

Security Audit

Report for DZapZapping

Date: June 12, 2025 **Version:** 1.0

Contact: contact@blocksec.com

Contents

Chapter 1 Introduction	1
1.1 About Target Contracts	1
1.2 Disclaimer	1
1.3 Procedure of Auditing	2
1.3.1 Software Security	2
1.3.2 DeFi Security	2
1.3.3 NFT Security	2
1.3.4 Additional Recommendation	3
1.4 Security Model	3
Chapter 2 Findings	4
2.1 DeFi Security	5
2.1.1 Incorrect functionality of the <code>withdrawERC721s()</code> and <code>revokeApprovals()</code> functions	5
2.1.2 Lack of check in function <code>_execute()</code> of the <code>DZapWallet</code> contract	5
2.1.3 Lack of check in function <code>_execute()</code> of the <code>DZapZapCore</code> contract	6
2.1.4 Potential inconsistency between approved amount and transferred amount	7
2.1.5 Potential replay attacks in the <code>execute()</code> and <code>deployWalletAndExecute()</code> functions	7
2.1.6 Lack of check on the fee ratio	8
2.1.7 Potential circumvention of referral fee charging mechanism	9
2.2 Recommendation	10
2.2.1 Add checks for the length of the <code>_label</code> parameter of the <code>getWalletByLabel()</code> function	10
2.2.2 Lack of minimum value check on quorum	10
2.2.3 Use <code>ReentrancyGuardUpgradeable</code> instead of <code>ReentrancyGuard</code> in the contract <code>DZapWallet</code>	11
2.3 Note	11
2.3.1 Potential centralization risks	11
2.3.2 Consistency of the <code>tokenAddress</code> and <code>tokenType</code>	11
2.3.3 Data verifications in the backend	12
2.3.4 Avoid duplications of output tokens of a <code>zapData</code>	12
2.3.5 Avoid overlaps of the input tokens and output tokens	13
2.3.6 Verify the <code>inputToken.fee</code> and <code>outputToken.fee</code> variables	14
2.3.7 Ensure the excessive approval of ERC20 tokens safe	15
2.3.8 Invoke <code>permit()</code> function only once if the allowance is insufficient	15
2.3.9 Ensure that there is no excessive ERC721 and ERC1155 tokens	16
2.3.10 Keep the <code>_DOMAIN_SEPARATOR</code> variable up-to-date	16
2.3.11 Verify the <code>_nativeValue</code> variable	17

Report Manifest

Item	Description
Client	DZapIO
Target	DZapZapping

Version History

Version	Date	Description
1.0	June 12, 2025	First release

Signature

About BlockSec BlockSec focuses on the security of the blockchain ecosystem and collaborates with leading DeFi projects to secure their products. BlockSec is founded by top-notch security researchers and experienced experts from both academia and industry. They have published multiple blockchain security papers in prestigious conferences, reported several zero-day attacks of DeFi applications, and successfully protected digital assets that are worth more than 14 million dollars by blocking multiple attacks. They can be reached at [Email](#), [Twitter](#) and [Medium](#).

Chapter 1 Introduction

1.1 About Target Contracts

Information	Description
Type	Smart Contract
Language	Solidity
Approach	Semi-automatic and manual verification

The target of this audit is the code repository ¹ of DZapZapping of DZapIO. DZapZapping allows users to create their wallets with unique labels and manages the transactions efficiently. Note this audit focuses on the smart contracts located in the /contract directory, excluding the following directories/files:

- /contract/adapters
- /contract/wallet/MinimalWallet.sol

The auditing process is iterative. Specifically, we would audit the commits that fix the discovered issues. If there are new issues, we will continue this process. The commit SHA values during the audit are shown in the following table. Our audit report is responsible for the code in the initial version ([Version 1](#)), as well as new code (in the following versions) to fix issues in the audit report.

Project	Version	Commit Hash
DZapZapping	Version 1	4f8e72b86accd30bc3ec7d3f9e9a2c47762e1eca
	Version 2	3d4c6f189758d6bcacda0e2adb059bc357314740

1.2 Disclaimer

This audit report does not constitute investment advice or a personal recommendation. It does not consider, and should not be interpreted as considering or having any bearing on, the potential economics of a token, token sale or any other product, service or other asset. Any entity should not rely on this report in any way, including for the purpose of making any decisions to buy or sell any token, product, service or other asset.

This audit report is not an endorsement of any particular project or team, and the report does not guarantee the security of any particular project. This audit does not give any warranties on discovering all security issues of the smart contracts, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit cannot be considered comprehensive, we always recommend proceeding with independent audits and a public bug bounty program to ensure the security of smart contracts.

The scope of this audit is limited to the code mentioned in Section 1.1. Unless explicitly specified, the security of the language itself (e.g., the solidity language), the underlying compiling toolchain and the computing infrastructure are out of the scope.

¹<https://github.com/DZapIO/DZapZappingContracts>

1.3 Procedure of Auditing

We perform the audit according to the following procedure.

- **Vulnerability Detection** We first scan smart contracts with automatic code analyzers, and then manually verify (reject or confirm) the issues reported by them.
- **Semantic Analysis** We study the business logic of smart contracts and conduct further investigation on the possible vulnerabilities using an automatic fuzzing tool (developed by our research team). We also manually analyze possible attack scenarios with independent auditors to cross-check the result.
- **Recommendation** We provide some useful advice to developers from the perspective of good programming practice, including gas optimization, code style, and etc.

We show the main concrete checkpoints in the following.

1.3.1 Software Security

- * Reentrancy
- * DoS
- * Access control
- * Data handling and data flow
- * Exception handling
- * Untrusted external call and control flow
- * Initialization consistency
- * Events operation
- * Error-prone randomness
- * Improper use of the proxy system

1.3.2 DeFi Security

- * Semantic consistency
- * Functionality consistency
- * Permission management
- * Business logic
- * Token operation
- * Emergency mechanism
- * Oracle security
- * Whitelist and blacklist
- * Economic impact
- * Batch transfer

1.3.3 NFT Security

- * Duplicated item
- * Verification of the token receiver
- * Off-chain metadata security

1.3.4 Additional Recommendation

- * Gas optimization
- * Code quality and style



Note The previous checkpoints are the main ones. We may use more checkpoints during the auditing process according to the functionality of the project.

1.4 Security Model

To evaluate the risk, we follow the standards or suggestions that are widely adopted by both industry and academy, including OWASP Risk Rating Methodology ² and Common Weakness Enumeration ³. The overall *severity* of the risk is determined by *likelihood* and *impact*. Specifically, likelihood is used to estimate how likely a particular vulnerability can be uncovered and exploited by an attacker, while impact is used to measure the consequences of a successful exploit.

In this report, both likelihood and impact are categorized into two ratings, i.e., *high* and *low* respectively, and their combinations are shown in Table 1.1.

Table 1.1: Vulnerability Severity Classification

Impact	High	High	Medium
	Low	Medium	Low
		High	Low
		Likelihood	

Accordingly, the severity measured in this report are classified into three categories: **High**, **Medium**, **Low**. For the sake of completeness, **Undetermined** is also used to cover circumstances when the risk cannot be well determined.

Furthermore, the status of a discovered item will fall into one of the following four categories:

- **Undetermined** No response yet.
- **Acknowledged** The item has been received by the client, but not confirmed yet.
- **Confirmed** The item has been recognized by the client, but not fixed yet.
- **Fixed** The item has been confirmed and fixed by the client.

²https://owasp.org/www-community/OWASP_Risk_Rating_Methodology

³<https://cwe.mitre.org/>

Chapter 2 Findings

In total, we found **seven** potential security issues. Besides, we have **three** recommendations and **eleven** notes.

- High Risk: 3
- Medium Risk: 2
- Low Risk: 2
- Recommendation: 3
- Note: 11

ID	Severity	Description	Category	Status
1	High	Incorrect functionality of the <code>withdrawERC721s()</code> and <code>revokeApprovals()</code> functions	DeFi Security	Fixed
2	High	Lack of check in function <code>_execute()</code> of the <code>DZapWallet</code> contract	DeFi Security	Fixed
3	High	Lack of check in function <code>_execute()</code> of the <code>DZapZapCore</code> contract	DeFi Security	Fixed
4	Medium	Potential inconsistency between approved amount and transferred amount	DeFi Security	Fixed
5	Medium	Potential replay attacks in the <code>execute()</code> and <code>deployWalletAndExecute()</code> functions	DeFi Security	Fixed
6	Low	Lack of check on the fee ratio	DeFi Security	Fixed
7	Low	Potential circumvention of referral fee charging mechanism	DeFi Security	Fixed
8	-	Add checks for the length of the <code>_label</code> parameter of the <code>getWalletByLabel()</code> function	Recommendation	Fixed
9	-	Lack of minimum value check on quorum	Recommendation	Fixed
10	-	Use <code>ReentrancyGuardUpgradeable</code> instead of <code>ReentrancyGuard</code> in the contract <code>DZapWallet</code>	Recommendation	Fixed
11	-	Potential centralization risks	Note	-
12	-	Consistency of the <code>tokenAddress</code> and <code>tokenType</code>	Note	-
13	-	Data verifications in the backend	Note	-
14	-	Avoid duplications of output tokens of a <code>zapData</code>	Note	-
15	-	Avoid overlaps of the input tokens and output tokens	Note	-
16	-	Verify the <code>inputToken.fee</code> and <code>outputToken.fee</code> variables	Note	-

17	-	Ensure the excessive approval of ERC20 tokens safe	Note	-
18	-	Invoke <code>permit()</code> function only once if the allowance is insufficient	Note	-
19	-	Ensure that there is no excessive ERC721 and ERC1155 tokens	Note	-
20	-	Keep the <code>_DOMAIN_SEPARATOR</code> variable up-to-date	Note	-
21	-	Verify the <code>_nativeValue</code> variable	Note	-

The details are provided in the following sections.

2.1 DeFi Security

2.1.1 Incorrect functionality of the `withdrawERC721s()` and `revokeApprovals()` functions

Severity High

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description The index `i` in both `withdrawERC20s()` and `revokeApprovals()` functions do not auto-increase. Thus, these two functions cannot work as expected.

```

93  function withdrawERC721s(IERC721[] calldata erc721s, uint256[] calldata ids, address[]
    calldata recipients) external onlyOwner {
94      uint256 length = ids.length;
95      for (uint256 i; i < length; ) {
96          _withdrawERC721s(erc721s[i], ids[i], recipients[i]);
97      }
98  }
```

Listing 2.1: contracts/wallet/MinimalWallet.sol

Impact Potential failures when withdrawing ERC20 tokens and revoking approvals.

Suggestion Revise the code logic properly.

Note This part of logic is out of our scope. However, it has a significant impact. Considering our responsibility, we believe it's important to report this issue.

2.1.2 Lack of check in function `_execute()` of the `DZapWallet` contract

Severity High

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description Function `_execute()` of the contract `DZapWallet` can delegatecall or call the other contracts. This results in two potential vulnerabilities. First, the attackers can delegatecall malicious implementation contract to modify some critical state variables (e.g., the `owner` variable of the `DZapWallet`). Second, the attackers can transfer all the tokens approved to the contract `DZapWallet` by calling specified token contracts (e.g., `transferfrom()` of ERC20 token).

```
80  function _execute(address _callTo, bytes memory _callData, uint256 _nativeValue, bool
    _isDelegateCall) private returns (bool success, bytes memory res) {
81      if (_callData.length != 0) {
82          if (_isDelegateCall) {
83              (success, res) = _callTo.delegatecall(_callData);
84              require(success, CallFailed(res));
85          } else {
86              (success, res) = _callTo.call{ value: _nativeValue }(_callData);
87              require(success, CallFailed(res));
88          }
89      }
90  }
```

Listing 2.2: contracts/wallet/DZapWallet.sol

Impact It may lead to loss of users' funds.

Suggestion Add proper check on the targets of call and delegatecall.

Note The target address of `delegatecall` has been whitelisted in the contract. The target address of `call` will be checked in the backend.

2.1.3 Lack of check in function `_execute()` of the `DZapZapCore` contract

Severity High

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description Function `_execute()` of the contract `DZapZapCore` can delegatecall or call the other contracts. This results in two potential vulnerabilities. First, the attackers can delegatecall malicious implementation contract to modify some critical state variables (e.g., the `verifier` variable of the `DZapZapCore`). Second, the attackers can transfer all the tokens approved to the contract `DZapZapCore` by calling specified token contracts (e.g., `transferfrom()` of ERC20 token).

```
173  function _execute(ZapData memory _zapData) private returns (bool success, bytes memory res) {
174      if (_zapData.callData.length != 0) {
175          if (_zapData.isDelegateCall) {
176              (success, res) = _zapData.callTo.delegatecall(_zapData.callData);
177              require(success, CallFailed(res));
178          } else {
179              (success, res) = _zapData.callTo.call{ value: _zapData.nativeValue }(_zapData.
                  callData);
180              require(success, CallFailed(res));
181          }
182      }
```

```
183 }
```

Listing 2.3: contracts/zap/DZapZapCore.sol

Impact Potential loss of funds.

Suggestion Add proper check on the targets of call and delegatecall.

Note The target address of delegate call has been whitelisted in the contract. The call target address will be checked in the backend.

2.1.4 Potential inconsistency between approved amount and transferred amount

Severity Medium

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description The function `depositErc20()` approves to the spender with a specified amount and transfers `_amount` tokens to the spender. However, the allowance (e.g., `allowanceAmount`) may be different from the actual transferred amount (i.e., `_amount`). If the approved amount is less, the transfer operation will revert. On the contrary, the approved allowance will not be used up and can be drained in the later operations.

```
13 function permit2ApproveAndTransfer(address _permit2, address _from, address _to, uint160
    _amount, address _token, bytes memory _data) internal {
14     if (_data.length != 0) {
15         (uint160 allowanceAmount, uint48 nonce, uint48 expiration, uint256 sigDeadline, bytes
            memory signature) = abi.decode(_data, (uint160, uint48, uint48, uint256, bytes));
16         IPermit2(_permit2).permit(msg.sender, IPermit2.PermitSingle(IPermit2.PermitDetails(
            _token, allowanceAmount, expiration, nonce), address(this), sigDeadline), signature
            );
17     }
18     IPermit2(_permit2).transferFrom(_from, _to, uint160(_amount), _token);
19 }
```

Listing 2.4: contracts/shared/libraries/LibPermit.sol

```
21 function permit(address _token, bytes memory _data) internal {
22     if (_data.length == 32 * 7) {
23         (bool success, ) = _token.call(bytes.concat(IERC20Permit.permit.selector, _data));
24         require(success, InvalidPermit());
25     } else revert InvalidPermitData();
26 }
```

Listing 2.5: contracts/shared/libraries/LibPermit.sol

Impact It may lead to loss of users' funds.

Suggestion Add proper checks.

2.1.5 Potential replay attacks in the `execute()` and `deployWalletAndExecute()` functions

Severity Medium

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description Function `execute()` and `deployWalletAndExecute()` verify the input parameter `_validatorSignatures` through the `_verify()` function before the cross contract invocation. However, there is no check on whether the `_validatorSignatures` has been used, allowing potential replay execution.

```

107  function execute(bytes32 _txId, uint256 _deadline, address _walletAddress, bytes calldata
      _callData, bytes calldata _validatorSignatures) external payable onlyAuthorizedExecutor
      whenNotPaused {
108      require(DZAP_FACTORY.isWalletDeployed(_walletAddress), WalletNotDeployed());
109      _verify(_txId, _walletAddress, _deadline, _callData, _validatorSignatures);
110      (bool success, bytes memory res) = _walletAddress.call{ value: msg.value }(_callData);
111      require(success, WalletExecutionCallFailed(res));
112      emit Executed(_txId, _walletAddress);
113  }

```

Listing 2.6: contracts/wallet/DZapExecutor.sol

```

115  function deployWalletAndExecute(bytes32 _txId, uint256 _deadline, address _userAddress, string
      memory _label, bytes calldata _callData, bytes calldata _validatorSignatures) external
      payable onlyAuthorizedExecutor whenNotPaused {
116      address walletAddress = DZAP_FACTORY.deploy(_userAddress, _label);
117      _verify(_txId, walletAddress, _deadline, _callData, _validatorSignatures);
118      (bool success, bytes memory res) = walletAddress.call{ value: msg.value }(_callData);
119      require(success, WalletExecutionCallFailed(res));
120      emit DeployAndExecuted(_txId, _userAddress, walletAddress);
121  }

```

Listing 2.7: contracts/wallet/DZapExecutor.sol

Impact Potential loss of funds.

Suggestion Check whether the `_validatorSignatures` is used.

2.1.6 Lack of check on the fee ratio

Severity Low

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description There are two ways to get registered as a referral. First, users can register through the function `registerAsReferral()`. The fee ratio is set as `defaultReferralNativeFeeShare` and `defaultReferralTokenFeeShare`, respectively. These two values are initialized in the `constructor` function of the `DZapZapCore` contract and updated in the `setDefaultReferralFee()` function. Second, users can also become a referral through the `addReferral()` functions, and the fee ratio is controllable through the `nativeFeeShare` and `tokenFeeShare` parameters. However, there are no checks (i.e., less than `_BPS_DENOMINATOR`) on all of the fee ratios, which may lead to unexpected errors.

```

73     constructor(address _owner, address _feeVault, address _verifier, address _permit2, uint96
        _defaultReferralNativeFeeShare, uint96 _defaultReferralTokenFeeShare, bytes32 _salt)
        Ownable(_owner) {
74         require(_verifier != address(0) && _permit2 != address(0), ZeroAddress());
75         require(_feeVault != address(0) && _feeVault != address(this), InvalidFeeVault());
76
77         feeVault = _feeVault;
78         permit2 = _permit2;
79         verifier = _verifier;
80         defaultReferralNativeFeeShare = _defaultReferralNativeFeeShare;
81         defaultReferralTokenFeeShare = _defaultReferralTokenFeeShare;
82         _DOMAIN_SEPARATOR = keccak256(abi.encode(_DOMAIN_TYPEHASH, keccak256(bytes("DZapVerifier"))
            , keccak256(bytes("1")), block.chainid, address(this), _salt));
83     }

```

Listing 2.8: contracts/zap/DZapZapCore.sol

```

87     function setDefaultReferralFee(uint96 _defaultReferralNativeFeeShare, uint96
        _defaultReferralTokenFeeShare) external onlyOwner {
88         defaultReferralNativeFeeShare = _defaultReferralNativeFeeShare;
89         defaultReferralTokenFeeShare = _defaultReferralTokenFeeShare;
90         emit DefaultReferralFeeSet(_defaultReferralNativeFeeShare, _defaultReferralTokenFeeShare);
91     }

```

Listing 2.9: contracts/zap/DZapZapCore.sol

```

115    function addReferral(address _referral, uint96 _nativeFeeShare, uint96 _tokenFeeShare)
        external onlyOwnerOrAdmin {
116        require(_referral != address(0), ZeroAddress());
117        referralFeeInfo[_referral] = ReferralFeeInfo({ nativeFeeShare: _nativeFeeShare,
            tokenFeeShare: _tokenFeeShare });
118        emit ReferralAdded(_referral);
119    }

```

Listing 2.10: contracts/zap/DZapZapCore.sol

```

49     uint256 private constant _BPS_DENOMINATOR = 1e6; // 4 basis points

```

Listing 2.11: contracts/zap/DZapZapCore.sol

Impact Unexpected errors due to incorrect fee settings.

Suggestion Check the `_defaultReferralNativeFeeShare`, `_defaultReferralTokenFeeShare`, `_nativeFeeShare`, and `_tokenFeeShare` variables.

2.1.7 Potential circumvention of referral fee charging mechanism

Severity Low

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description In the `zap()` function of the `DZapZapCore` contract, the `referral` fee is sent to the `_referral` address. Thus, users can circumvent the fee charge mechanism by set the `_referral` address to themselves.

```
148 function zap(bytes32 _transactionId, bytes calldata _data, bytes calldata _signature, address
    _referral, InputErc20Tokens[] calldata _inputTokens, address[] calldata _sweepDust,
    address _dustReciever) external payable nonReentrant refundExcessNative(_dustReciever) {
149     require(_dustReciever != address(0), NoTransferToNullAddress());
150     _handleVerification(_transactionId, _referral, _data, _signature);
151     _handleErcDeposits(_inputTokens);
152     _handleZap(_data, _referral);
153     _handleSweepTokens(_sweepDust, _dustReciever);
154     emit Zapped(msg.sender, _transactionId);
155 }
```

Listing 2.12: contracts/zap/DZapZapCore.sol

Impact The referral fee charging mechanism can be circumvented.

Suggestion Ensure the `msg.sender` variable is different from the `_referral` address.

2.2 Recommendation

2.2.1 Add checks for the length of the `_label` parameter of the `getWalletByLabel()` function

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description Based on the code of `deploy()` and `getOrDeploy()`, the length of the `_label` of a wallet can never be 0. Thus, the `getWalletByLabel()` function should also check the length of the input `_label` parameter.

Suggestion Add validation checks on the length of the parameter `_label`.

2.2.2 Lack of minimum value check on quorum

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description The function `setQuorum()` does not have a minimum value check of quorum.

```
83 function setExecutorWhitelisting(address[] memory _executorsArr, bool _whitelisted) external
    onlyOwner {
84     _setExecutors(_executorsArr, _whitelisted);
85     emit ExecutorWhitelistingUpdated(_executorsArr, _whitelisted);
86 }
```

Listing 2.13: contracts/wallet/DZapExecutor.sol

Impact The minimum number of valid signatures required might be 1, which decreases safety.

Suggestion Add a minimum value of quorum.

2.2.3 Use ReentrancyGuardUpgradeable instead of ReentrancyGuard in the contract DZapWallet

Status Fixed in [Version 2](#)

Introduced by [Version 1](#)

Description The contract DZapWallet is an implementation contract. Thus, the init value of `NOT_ENTERED` set in the constructor of ReentrancyGuard will not apply to the proxy.

```
35contract DZapWallet is Initializable, MinimalWallet, ReentrancyGuard, IDZapWallet {
```

Listing 2.14: contracts/wallet/DZapWallet.sol

Impact The init value of `NOT_ENTERED` set in the constructor of ReentrancyGuard will not apply to the proxy.

Suggestion Use ReentrancyGuardUpgradeable instead of ReentrancyGuard in the contract DZapWallet.

2.3 Note

2.3.1 Potential centralization risks

Introduced by [Version 1](#)

Description Several protocol roles (e.g., the admin role) could conduct privileged operations, which introduces potential centralization risks. If the private keys of the privileged accounts are lost or maliciously exploited, it could pose a significant risk to the protocol.

2.3.2 Consistency of the tokenAddress and tokenType

Introduced by [Version 1](#)

Description The verifier should make sure the `tokenAddress` and `tokenType` are consistent in function `_processInputTokens()`.

```
25 struct InputToken {
26     TokenType tokenType;
27     InputTransferType transferType;
28     address tokenAddress;
29     address approveTo;
30     uint256 amount;
31     uint256 tokenId;
32     uint256 fee;
33 }
```

Listing 2.15: contracts/zap/Types.sol

```
314 function _processInputTokens(address _referral, uint256 _inputCount, ZapData memory _zapData,
    InputToken[] memory _inputTokens, ReferralFeeInfo memory _referralFee) private returns (
    uint256 totalNativeFeeAmount, uint256 totalReferralNativeFeeAmount) {
315     uint256 length = _inputCount + _zapData.inputLength;
316     for (uint256 i = _inputCount; i < length; ++i) {
```

```
317     InputToken memory inputToken = _inputTokens[i];
318
319     if (inputToken.tokenType == TokenType.NATIVE) {
320         (uint256 totalFeeAmount, uint256 referralFeeAmount) = _handleNativeInput(inputToken
            , inputToken.fee, _referralFee.tokenFeeShare);
321
322         totalNativeFeeAmount += totalFeeAmount;
323         totalReferralNativeFeeAmount += referralFeeAmount;
324     } else if (inputToken.tokenType == TokenType.ERC20) _handleErc20Input(inputToken,
        inputToken.fee, _referralFee.tokenFeeShare, _referral);
325     else if (inputToken.tokenType == TokenType.ERC721) _handleErc721Input(inputToken);
326     else if (inputToken.tokenType == TokenType.ERC1155) _handleErc1155Input(inputToken);
327 }
328 }
```

Listing 2.16: contracts/zap/DZapZapCore.sol

2.3.3 Data verifications in the backend

Introduced by [Version 1](#)

Description The functionality and security of this project rely heavily on the data verification in the backend. The issues (i.e., Issue-2, and Issue-3) are mitigated through backend verifications. Therefore, it is crucial to ensure that key fields (e.g., call-to addresses, fees, and executors) are thoroughly verified in the backend.

Moreover, the function `crossZap()` enables cross-chain asset transfers by bridging the user's funds from the source chain to the user's `DZapWallet` on the destination chain with a specified `vHash`, which represents the user intention on the destination chain. The backend monitors invocation of function `crossZap()` off-chain, verifies that the correct amount of token is bridged to the correct user's wallet address. After that, the `EXECUTOR` invokes the function `execute()` in the `DZapExecutor` contract to execute the user's intention on destination chain. This flow relies heavily on secure backend verification of invocation of function `crossZap()` and `vHash` to prevent mismatches in amounts or incorrect execution on destination chain.

2.3.4 Avoid duplications of output tokens of a zapData

Introduced by [Version 1](#)

Description When the output tokens of a `zapData` have duplications, the check of `returnAmount >= _outputToken.minReturn` can be bypassed. This is because the `returnAmount` will count more due to duplications. Therefore, the project should ensure there is no duplication of output tokens of a `zapData` in the backend.

```
274     function _handleERC20Output(OutputToken memory _outputToken, address _recipient, uint256
        _initialBalance, uint256 _tokenFee, uint256 _referralFee, address _referralAddress)
        private returns (uint256 totalFeeAmount, uint256 referralFeeAmount) {
275         uint256 returnAmount = LibAsset.getBalance(_outputToken.tokenAddress, _recipient) -
            _initialBalance;
276         require(returnAmount >= _outputToken.minReturn, InvalidReturnAmount(returnAmount,
            _outputToken.minReturn));
```

```
277
278     if (_tokenFee != 0) {
279         require(_outputToken.transferType != OutputTransferType.DirectTransferToRecipient,
                InvalidOutputType());
280         (totalFeeAmount, referralFeeAmount) = _getTotalAnReferralFeeAmount(returnAmount,
                _tokenFee, _referralFee);
281     }
282     uint256 amount = returnAmount - totalFeeAmount;
283
284     if (_outputToken.transferType == OutputTransferType.ReceiveAndTransfer) LibAsset.
        transferERC20(_outputToken.tokenAddress, _outputToken.recipient, amount);
285
286     _transferTokenFee(_outputToken.tokenAddress, _referralAddress, totalFeeAmount,
        referralFeeAmount);
287 }
```

Listing 2.17: contracts/zap/DZapZapCore.sol

2.3.5 Avoid overlaps of the input tokens and output tokens

Introduced by [Version 1](#)

Description When input tokens overlap with output tokens in a zapData, users may receive fewer output tokens than expected. This occurs because the `_handleZap()` function calculates the initial balances of the output tokens after processing the input tokens but before executing `_execute()`. Specifically, the input tokens are first transferred to the contract and then approved for the assetTo when transferType is `ApproveForSpender`. If the transferType for the output tokens is `ReceiveAndTransfer`, their initial balances inadvertently include the balance of the input tokens when they overlap. Therefore, the project should ensure that there are no overlaps of the input tokens and output tokens in the backend.

```
213 function _getOutputTokensInitialBalance(uint256 _calldataLength, uint256 _outputLength,
    uint256 _nativeValueToTransfer, uint256 _outputCount, OutputToken[] memory _outputTokens)
    private view returns (uint256[] memory) {
214     if (_calldataLength != 0) {
215         uint256 outputEndIndex = _outputCount + _outputLength;
216         uint256[] memory initialOutputBalances = new uint256[](_outputLength);
217         uint256 index;
218         for (uint256 i = _outputCount; i < outputEndIndex; ++i) {
219             OutputToken memory outputToken = _outputTokens[i];
220             address recipient = _getRecipient(outputToken);
221
222             if (outputToken.tokenType == TokenType.ERC20) {
223                 initialOutputBalances[index++] = LibAsset.getErc20Balance(outputToken.
                    tokenAddress, recipient);
224             } else if (outputToken.tokenType == TokenType.NATIVE) {
225                 initialOutputBalances[index++] = recipient == address(this) ? LibAsset.
                    selfNativeBalance() - _nativeValueToTransfer : LibAsset.getNativeBalance(
                    recipient);
226             } else if (outputToken.tokenType == TokenType.ERC1155) {
227                 initialOutputBalances[index++] = LibAsset.getBalanceOfERC1155(outputToken.
                    tokenAddress, recipient, outputToken.tokenId);
```



```
228     }
229 }
230
231     return initialOutputBalances;
232 }
233 }
```

Listing 2.18: contracts/zap/DZapZapCore.sol

2.3.6 Verify the `inputToken.fee` and `outputToken.fee` variables

Introduced by [Version 1](#)

Description In the `_processInputTokens()` and `_processOutputTokens()` functions, the fee calculations depend on the `inputToken.fee` and `outputToken.fee` variables, which are both extracted from the `_data` parameter of the `_handleZap()` function. The project should verify these two variables to avoid fee charge circumvention in the backend.

```
314 function _processInputTokens(address _referral, uint256 _inputCount, ZapData memory _zapData,
    InputToken[] memory _inputTokens, ReferralFeeInfo memory _referralFee) private returns (
    uint256 totalNativeFeeAmount, uint256 totalReferralNativeFeeAmount) {
315     uint256 length = _inputCount + _zapData.inputLength;
316     for (uint256 i = _inputCount; i < length; ++i) {
317         InputToken memory inputToken = _inputTokens[i];
318
319         if (inputToken.tokenType == TokenType.NATIVE) {
320             (uint256 totalFeeAmount, uint256 referralFeeAmount) = _handleNativeInput(inputToken
                , inputToken.fee, _referralFee.tokenFeeShare);
321
322             totalNativeFeeAmount += totalFeeAmount;
323             totalReferralNativeFeeAmount += referralFeeAmount;
324         } else if (inputToken.tokenType == TokenType.ERC20) _handleErc20Input(inputToken,
            inputToken.fee, _referralFee.tokenFeeShare, _referral);
325         else if (inputToken.tokenType == TokenType.ERC721) _handleErc721Input(inputToken);
326         else if (inputToken.tokenType == TokenType.ERC1155) _handleErc1155Input(inputToken);
327     }
328 }
```

Listing 2.19: contracts/zap/DZapZapCore.sol

```
330 function _processOutputTokens(address _referral, uint256 _outputCount, ZapData memory _zapData
    , OutputToken[] memory _outputTokens, ReferralFeeInfo memory _referralFee, uint256[]
    memory _initialOutputBalances) private returns (uint256 totalNativeFeeAmount, uint256
    totalReferralNativeFeeAmount) {
331     uint256 length = _outputCount + _zapData.outputLength;
332     uint256 index;
333     for (uint256 i = _outputCount; i < length; ++i) {
334         OutputToken memory outputToken = _outputTokens[i];
335         address recipient = _getRecipient(outputToken);
336
337         if (outputToken.tokenType == TokenType.ERC20) _handleERC20Output(outputToken, recipient
            , _initialOutputBalances[index++], outputToken.fee, _referralFee.tokenFeeShare,
            _referral);
```

```
338         else if (outputToken.tokenType == TokenType.NATIVE) {
339             (uint256 totalFeeAmount, uint256 referralFeeAmount) = _handleNativeOutput(
                outputToken, recipient, _initialOutputBalances[index++], outputToken.fee,
                _referralFee.tokenFeeShare);
340
341             totalNativeFeeAmount += totalFeeAmount;
342             totalReferralNativeFeeAmount += referralFeeAmount;
343         } else if (outputToken.tokenType == TokenType.ERC721) _handleERC721Output(outputToken,
            recipient);
344         else if (outputToken.tokenType == TokenType.ERC1155) _handleERC1155Output(outputToken,
            recipient, _initialOutputBalances[index++]);
345     }
346 }
```

Listing 2.20: contracts/zap/DZapZapCore.sol

2.3.7 Ensure the excessive approval of ERC20 tokens safe

Introduced by Version 1

Description The function `_handleZap()` revokes the approval of ERC1155 tokens and does not revoke the approval of ERC721 and ERC20 tokens. Revoking approval of ERC721 tokens is needless since when an NFT is transferred, the approval is automatically cleared by the `_update()` function. Therefore, the project should block attack surfaces targeting the excessive approval of ERC20 tokens.

```
164     function _getRecipient(OutputToken memory outputToken) private view returns (address) {
165         return outputToken.transferType == OutputTransferType.ReceiveAndTransfer ? address(this) :
            outputToken.recipient;
166     }
```

Listing 2.21: contracts/zap/DZapZapCore.sol

2.3.8 Invoke `permit()` function only once if the allowance is insufficient

Introduced by Version 1

Description In the `LibAsset` contract, the `depositErc20()` function facilitates the token permit process. The `permit()` function of the `ERC20Permit` contract allows anyone to use a valid permit signature on the original signer's behalf. The permit signature becomes invalid after being consumed by the `permit()` function once. An attacker can monitor a `depositErc20()` transaction in the memory pool, copying the signature, and then executing a permit transaction to front-run the original `depositErc20()` transaction. This action consumes the nonce linked to the signature, rendering it invalid. Therefore, the project should increase the allowance through the `permit()` function only once if the allowance is insufficient.

```
21     function permit(address _token, bytes memory _data) internal {
22         if (_data.length == 32 * 7) {
23             (bool success, ) = _token.call(bytes.concat(IERC20Permit.permit.selector, _data));
24             require(success, InvalidPermit());
25         } else revert InvalidPermitData();
```

```
26 }
```

Listing 2.22: contracts/shared/libraries/LibPermit.sol

2.3.9 Ensure that there is no excessive ERC721 and ERC1155 tokens

Introduced by Version 1

Description The function `_handleSweepTokens()` does not sweep ERC721 and ERC1155 tokens. Therefore, the project should ensure that there are no ERC721 and ERC1155 tokens leftover after the Zap process in the backend.

```
364 function _handleSweepTokens(address[] calldata _sweepErc20, address _dustReceiver) internal {
365     uint256 length = _sweepErc20.length;
366     for (uint256 i = 0; i < length; ++i) {
367         address tokenAddress = _sweepErc20[i];
368         uint256 balance = LibAsset.getBalance(tokenAddress, address(this));
369         if (balance != 0) LibAsset.transferERC20(tokenAddress, _dustReceiver, balance);
370     }
371 }
```

Listing 2.23: contracts/zap/DZapZapCore.sol

2.3.10 Keep the `_DOMAIN_SEPARATOR` variable up-to-date

Introduced by Version 1

Description In the event of a hard fork, the `chainid` may be changed. If a contract continues to use the old `chainID`, the `_DOMAIN_SEPARATOR` becomes invalid. Therefore, the project should make sure that the `chainID` in the `_DOMAIN_SEPARATOR` variable is up-to-date.

```
54 constructor(address _newOwner, address _dZapFactory, uint8 _quorum, address[] memory
    _executorsToAdd, address[] memory _validatorsToAdd) Ownable(_newOwner) {
55     _DOMAIN_SEPARATOR = keccak256(abi.encode(_DOMAIN_TYPEHASH, keccak256(bytes("DZapExecutor")),
        , keccak256(bytes("1")), block.chainid, address(this)));
56     require(_quorum > 0, QuorumTooLow());
57     DZAP_FACTORY = IDZapWalletFactory(_dZapFactory);
58     quorum = _quorum;
59     _setExecutors(_executorsToAdd, true);
60     _setValidators(_validatorsToAdd, true);
61 }
```

Listing 2.24: contracts/wallet/DZapExecutor.sol

```
125 function _verify(bytes32 _txId, address _walletAddress, uint256 _deadline, bytes calldata
    _data, bytes calldata _validatorSignatures) private view {
126     require(_deadline >= block.timestamp, SigDeadlineExpired());
127     bytes32 msgHash = keccak256(abi.encode(_VALIDATOR_SIGNED_DATA_TYPEHASH, _txId, msg.sender,
        _walletAddress, _deadline, keccak256(_data)));
128     bytes32 digest = keccak256(abi.encodePacked("\x19\x01", _DOMAIN_SEPARATOR, msgHash));
129     LibValidator.verifyValidatorSigs(_validators, _validatorSignatures, digest, quorum);
130 }
```

Listing 2.25: contracts/wallet/DZapExecutor.sol

2.3.11 Verify the `_nativeValue` variable

Introduced by Version 1

Description The `executeByExecutor()` and `execute()` functions in the `DZapWallet` contract conduct the cross-contract invocation with `_nativeValue` parameter. The project should verify the `_nativeValue` variable to avoid excessive consumption of native tokens in the backend.

```
60 function executeByExecutor(bytes32 _txId, address[] calldata _callTo, bytes[] calldata
    _callData, uint256[] calldata _nativeValue, bool[] calldata _isDelegateCall) external
    payable onlyAuthorizedExecutor nonReentrant {
61     uint256 length = _callTo.length;
62     for (uint256 i; i < length; ++i) {
63         _execute(_callTo[i], _callData[i], _nativeValue[i], _isDelegateCall[i]);
64     }
65
66     emit Executed(_txId);
67 }
```

Listing 2.26: contracts/wallet/DZapWallet.sol

```
69 function execute(bytes32 _txId, address[] calldata _callTo, bytes[] calldata _callData,
    uint256[] calldata _nativeValue, bool[] calldata _isDelegateCall) external payable
    onlyOwner nonReentrant {
70     uint256 length = _callTo.length;
71     for (uint256 i; i < length; ++i) {
72         _execute(_callTo[i], _callData[i], _nativeValue[i], _isDelegateCall[i]);
73     }
74
75     emit Executed(_txId);
76 }
```

Listing 2.27: contracts/wallet/DZapWallet.sol

```
80 function _execute(address _callTo, bytes memory _callData, uint256 _nativeValue, bool
    _isDelegateCall) private returns (bool success, bytes memory res) {
81     if (_callData.length != 0) {
82         if (_isDelegateCall) {
83             (success, res) = _callTo.delegatecall(_callData);
84             require(success, CallFailed(res));
85         } else {
86             (success, res) = _callTo.call{ value: _nativeValue }(_callData);
87             require(success, CallFailed(res));
88         }
89     }
90 }
```

Listing 2.28: contracts/wallet/DZapWallet.sol

